ARUNAI ENGINEERING COLLEGE

(Autonomous)
Velu Nagar, Thiruvannamalai-606603
www.arunai.org

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

BACHELOR OF TECHNOLOGY

2025 – 2026

(EVEN SEMESTER)

THIRD YEAR

CCS339-CRYPTOCURRENCY AND BLOCKCHAIN TECHNOLOGIES

ARUNAI ENGINEERING COLLEGE

(Autonomous)
TIRUVANNAMALAI – 606 603

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

Certified that this is a bonafide record of work done by

Name :

University Reg.No :

Semester :

Branch :

Year :

Staff-in-Charge **Head of the Department**

Submitted for the _____

Practical Examination held on _____

Internal Examiner **External Examiner**

S.NO	DATE	List of Experiments	Pg. No	Signature
1		Install and understand Docker container, Node.js, Java and Hyperledger Fabric, Ethereum and perform necessary software installation on local machine/create instance on cloud to run.		
2		Create and deploy a blockchain network using Hyperledger Fabric SDK for Java Set up and initialize the channel, install and instantiate chain code, and perform invoke and query on your blockchain network.		
3		Interact with a blockchain network. Execute transactions and requests against blockchain		
4		Deploy an asset-transfer app using blockchain. Learn app development within a Hyperledger Fabric network.		
5		Fitness Club rewards using Hyperledger Fabric		
6		Car auction network using the Hyperledger fabric node SDK		

Ex.No: 1 Date:	Install and understand Docker container, Node.js, Java and Hyperledger Fabric, Ethereum and perform necessary software installation on local machine/create instance on cloud to run.

Aim

To Install and understand Docker container, Node.js, Java and Hyperledger Fabric, Ethereum and perform necessary software installation on local machine

Docker

Docker is an OS-level virtualization software platform that helps users in building and managing applications in the Docker environment with all its library dependencies. Docker is considered a better alternative to a virtual machine.

The main components of Docker are:

- Docker Engine: The core component of Docker that manages containers. It includes a server and a REST API that allows users to interact with the Docker daemon.
- Docker Images: A Docker image is a lightweight, standalone, and executable package that contains everything needed to run an application, including the code, runtime, libraries, environment variables, and configuration files.
- Docker Container: A container is an instance of a Docker image, running in an isolated environment. Containers are lightweight, start quickly, and use the host operating system's kernel to run the application.

1. Install and understand Docker container, Node.js, Java

To install the Docker container, Node.js, Java on Ubuntu**.**

Step 1: Install and understand Docker container, Node.js, Java

Step 2: Confirm the latest versions of both Docker and Docker Compose executables were installed.

Step 3: Start the docker service

Step 4: Pull a CentOS image from docker hub and run the CentOS container.

Step 5: Run the CentOS container.

Install Docker Desktop on Ubuntu

Step 1: Docker installation On Ubuntu

Install docker on Ubuntu box, first let us update the packages.

```
sudo apt-get update
```

Step 2: Install Docker

Install the latest version of Docker if it is not already installed.

```
sudo apt-get -y install docker-compose
```

Step 3: Confirm the latest versions of both Docker and Docker Compose executables were installed.

```
docker --version
```

Docker version 20.10.12, build 20.10.12-0ubuntu2~20.04.1

docker-compose --version

docker-compose version 1.25.0, build

Step 4: Make sure the Docker daemon is running.

sudo systemctl start docker

Optional: If you want the Docker daemon to start when the system starts, use the following:

sudo systemctl enable docker

Step 5: Start the docker service.

sudo service docker start

It says your job is already running. Docker has been successfully installed.

Step 6: Pull a CentOS image from docker hub and run the CentOS container.

sudo docker pull centos

Step 7: Run the CentOS container

sudo docker run -it centos

First installed docker on Ubuntu, after that we have pulled a CentOS image from docker hub and using that image, we have successfully built a CentOS container.

2. Install and understand Hyperledger Fabric and Ethereum Aim:

To create a cloud environment instance install the Hyperledger Fabric and Ethereum.

Procedure:

Step 1: Install curl, git
Step 2: Install Docker
Step 3: Install Golang
Step 4: Install Hyperledger Fabric
Step 5: Update the repositories
Step 6: Install Ethereum
Step 7: Install the Test RPC

Step 1: Install curl, git

Install the latest version of git and curl if it is not already installed.

sudo apt update
sudo apt-get install git
sudo apt-get install curl

Step 2: Install Docker

Install the latest version of Docker if it is not already installed.

sudo apt-get -y install docker-compose
Once installed, confirm that the latest versions of both Docker and Docker Compose executables were installed.

docker –version
Docker version 20.10.12, build 20.10.12-0ubuntu2~20.04.1

docker-compose –version
docker-compose version 1.25.0, build

Make sure the Docker daemon is running.

sudo systemctl start docker
Optional: If you want the Docker daemon to start when the system starts, use the following:

sudo systemctl enable docker
Add your user to the Docker group.

sudo usermod -a -G docker <username>

Step 3: Install Golang

curl -O https://dl.google.com/go/go1.18.3.linux-amd64.tar.gz
tar xvf go1.18.3.linux-amd64.tar.gz
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin

Now, you have to open and edit you **bashrc **file. In most cases, the bashrc is a hidden file that lives in your home directory.

nano ~/.bashrc

Add following two lines, at the end of the file:

export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin

Create new directory where you will install Hyperledger Fabric

mkdir fabric-network

Step 4: Install Hyperledger Fabric

To download a specific release, pass a version identifier for Fabric and Fabric CA Docker images. The command below demonstrates how to download the latest production releases - Fabric v2.4.4 and Fabric CA v1.5.3

curl -sSL https://bit.ly/2ysbOFE | bash -s -- <fabric_version> <fabric-ca_version>

curl -sSL https://bit.ly/2ysbOFE | bash -s -- 2.4.4 1.5.3

Download the latest release of Fabric samples, docker images, and binaries.

curl -sSL https://bit.ly/2ysbOFE | bash -s

Now you have installed Hyperledger Fabric with all dependencies.

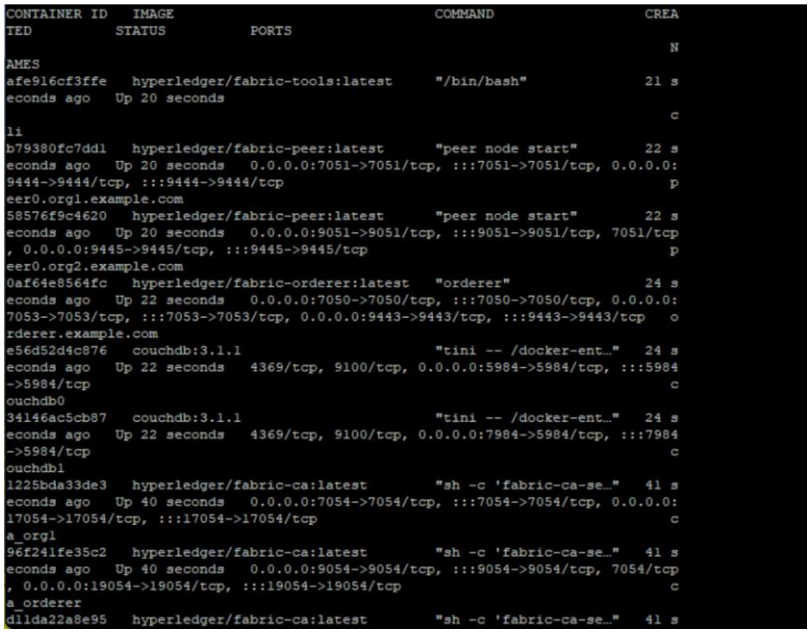
To test the our installation, we will run test network that comes with fabric-samples

cd fabric-samples/test-network
./network.sh up createChannel -ca -c mychannel -s couchdb

The above command will create a channel(mychannel) with certificate Authorities and bring up couch-db as state database.

Using following Docker command we can check all running containers

docker ps -a
You should be able to see the containers up and running.



You have installed and setup Hyperledger Fabric.

Step 5: Update the repositories

.sudo apt update

Step 6: Install ethereum

sudo apt install ethereum
You can check the installation by using the geth version command

Step 7: Install the Test RPC

Test RPC is an Ethereum node emulator implemented in NodeJS. The purpose of this Test RPC is to easily start the Ethereum node for test and development purposes.

sudo npm install -g ethereumjs-testrpc

It created 10 test accounts and displays respective private keys. We can use these accounts for our testing purposes. By default the Test

Result:

The installation of Docker, Node.js, Java, Hyperledger Fabric, and Ethereum, along with the necessary software setup on the local machine or cloud instance, was successfully completed.

Ex.No: 2 Date:	Create and deploy a blockchain network using Hyperledger Fabric SDK for Java Set up and initialize the channel, install and instantiate chain code, and perform invoke and query on your blockchain network.

Aim:

To create and deploy a blockchain network using Hyperledger Fabric SDK for Java Set up and initialize the channel**.**

Block chain network

Blockchain is a type of shared database that differs from a typical database in the way it stores information. Blockchains store data in blocks linked together via cryptography.

The Hyperledger Fabric Docker images and samples, you can deploy a test network by using scripts that are provided in the fabric-samples repository. The network is meant to be used only as a tool for education and testing and not as a model for how to set up a network

- It includes two peer organizations and an ordering organization.
- For simplicity, a single node Raft ordering service is configured.
- To reduce complexity, a TLS Certificate Authority (CA) is not deployed. All certificates are issued by the root CAs.

- **Prerequisite software:** The base layer needed to run the software, for example,
  - Git
  - Curl
  - Docker
  - Jq
  - Docker
- Contract APIs: To develop smart contracts executed on a Fabric Network.
- Application SDKs: To develop your blockchain application.
- The Application: Blockchain application will utilize the Application SDKs to call smart contracts running on a Fabric network.

1. Create and deploy a blockchain network using Hyperledger Fabric SDK for Java Set up and initialize the channel.

Procedure:

Step 1: Install Fabric and Fabric Samples Step 2:
Download fabric samples
Step 3: Pull the docker containers
Step 4: Navigate to test network directory

Step 5: Remove any containers or artifacts Step
6: Up the network
Step 7: Verify the containers Step
8: Create a channel

sudo apt update

sudo apt-get install git

sudo apt install -y jq

If the Jq installtion Error arises and If lock files is there need to remove lock by

sudo rm /var/lib/apt/lists/lock

sudo rm /var/cache/apt/archives/lock

sudo rm /var/lib/dpkg/lock

sudo dpkg --configure -a

Kill running process

Find Process id by ps then install

sudo apt update

sudo apt install -y jq

Step 1: Install Fabric and Fabric Samples

mkdir fabric //*used to make a new directory*//

cd fabric //change directory//

Step 2: Download fabric samples

Before you can run the test network, you need to install Fabric Samples in your environment.

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh && chmod +x install-fabric.sh

//*Client Url: is a command line tool that enables data transfer over various network protocols*// **Step 3: Pull the docker containers**
./install-fabric.sh

Step 4: Navigate to test network directory

cd fabric-samples/test-network

Step 5: Remove any containers or artifacts

./network.sh down

Step 6: Up the network

./network.sh up

Step 7: Verify the containers

docker ps -a

Step 8: Create a channel

./network.sh createChannel -c mychannel

//*mychannel is the channel name//*

For Step 6 And Step 8: Up and create channel in single step (already done)

./network.sh up createChannel

2 Install and Instantiate chain code, and Perform invoke and Query on your blockchain network.

To install and Instantiate chain code, and perform invoke and Query on your blockchain network

Procedure:

Step 1: Deploy chaincode on peers and channel

Step 2: Set the path for peer binary and config for core.yaml

Step 3: Set the environment variables to operate Peer as Org1

Step 4: Command to initialize the ledger with assets

Step 5: Query the ledger

Step 1: Deploy chaincode on peers and channel

./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-typescript -ccl typescript

//the deployCC ***subcommand will install the asset-transfer (basic) chaincode on*** peer0.org1.example.com ***and*** peer0.org2.example.com ***and then deploy the chaincode on the channel specified using the channel flag (or*** mychannel ***if no channel is specified). "-ccn basic" This specifies the name of the chaincode as "basic". If you are deploying a chaincode for the first time, the script will install the chaincode dependencies. You can use the language flag,*** -l ***, to install the Go, typescript or javascript versions of the chaincode. You can find the asset-transfer (basic) chaincode in the*** asset-transfer-basic ***folder of the*** fabric-samples ***directory.//***

Interacting with the network

Step 2: Set the path for peer binary and config for core.yaml

export PATH=${PWD}/../bin:$PATH

// Make sure that you are operating from the test-network ***directory. If you followed the instructions to install the Samples, Binaries and Docker Images, You can find the*** peer ***binaries in the*** bin ***folder of the*** fabric-samples ***repository. Use the following command to add those binaries to your CLI Path: The command export PATH=${PWD}/../bin: $PATH is used to temporarily modify the system's PATH environment variable. //***

export FABRIC_CFG_PATH=$PWD/../config/

//You also need to set the FABRIC_CFG_PATH *to point to the* core.yaml *file in the* fabric-samples *repositor The FABRIC_CFG_PATH environment variable is used by Hyperledger Fabric tools and commands to locate the network configuration files needed for various operations, such as creating channels, joining peers, and installing chaincode.y.//*

Step 3: Set the environment variables to operate Peer as Org1

export CORE_PEER_TLS_ENABLED=true *// used to enable Transport Layer Security (TLS) communication for the peer. When set to true, the peer will use TLS to secure its communication //*

export CORE_PEER_LOCALMSPID="Org1MSP"

// This sets the environment variable CORE_PEER_LOCALMSPID to the value "Org1MSP". CORE_PEER_LOCALMSPID is the Local Membership Service Provider Identifier for the peer. It identifies the peer's Membership Service Provider (MSP), which contains the identity information of the organization. //

export CORE_PEER_TLS_ROOTCERT_FILE=$ {PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

// This sets the environment variable CORE_PEER_TLS_ROOTCERT_FILE to the path of the TLS root certificate file for "Org1"'s peer.

The TLS root certificate is used to establish the trust relationship between the peer and other components in the network when using TLS communication. //

export CORE_PEER_MSPCONFIGPATH=$ {PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

// This sets the environment variable CORE_PEER_MSPCONFIGPATH to the path of the MSP (Membership Service Provider) configuration for the peer's administrative identity. The MSP configuration contains cryptographic material and identity information used by the peer to participate in the network. //

// This sets the environment variable CORE_PEER_ADDRESS to the value localhost:7051//

Step 4: Command to initialize the ledger with assets

Chaincodes are invoked when a network member wants to transfer or change an asset on the ledger. Use the following command to change the owner of an asset on the ledger by invoking the asset-transfer (basic) chaincode:

peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$

{PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" --peerAddresses localhost:9051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" -c '{"function":"InitLedger","Args":[]}'

// This commands command is used to initialize the ledger of the chaincode named "basic" on the "mychannel" channel by invoking the "InitLedger" function on both Org1's and Org2's peers. It ensures that the transaction is encrypted using TLS and that the peers can verify each other's TLS certificates for secure communication. Because the endorsement policy for the asset-transfer (basic) chaincode requires the transaction to be signed by Org1 and Org2, the chaincode invoke command needs to target both `peer0.org1.example.com` ***and*** `peer0.org2.example.com` ***using the*** `--peerAddresses` ***flag. Because TLS is enabled for the network, the command also needs to reference the TLS certificate for each peer using the*** `--tlsRootCertFiles` ***flag. //***

Step 5: Query the ledger

Run the following command to get the list of assets that were added to your channel ledger:

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

The output of the command will display the result of the "GetAllAssets" function query, showing the assets retrieved from the ledger.

Result:

Thus, the blockchain network using Hyperledger Fabric SDK for Java is set up successfully, the channel is initialized, and the chaincode is installed, invoked, and queried successfully.

Ex.No: 3 Date:	Interact with a blockchain network. Execute transactions and requests against blockchain

Aim:

To create a simple Java program that simulates interacting with a blockchain network by executing transactions, adding blocks, and validating the blockchain's structure and rules.

Algorithm :

1. **Blockchain Setup:**
   - Create a Block class to represent the individual blocks in the blockchain.
   - Each block contains:
     - Index
     - Timestamp
     - Data (transactions)
     - Previous Hash
     - Hash (computed for the current block)
2. **Block Creation:**
   - Add a method to create and append a new block to the blockchain.
3. **Transaction Execution:**
   - Create transactions with a sender, recipient, and amount.
   - Add each transaction to a list before a new block is mined.
4. **Proof of Work (Optional for Consensus):**
   - Implement a simple Proof of Work (PoW) method (finding a hash with a required number of leading zeros).
5. **Transaction Validation:**
   - Before executing a transaction, verify if the sender has enough balance to complete the transaction.
6. **Hash Calculation:**
   - Compute a hash for each block to ensure integrity and link blocks securely.

Program

```
import java.util.ArrayList;
import java.util.List;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Date;

public class Blockchain {

   private List<Block> chain;
   private List<Transaction> currentTransactions;

   public Blockchain() {
      this.chain = new ArrayList<>();
      this.currentTransactions = new ArrayList<>();
```

```
    // Create Genesis Block
    this.newBlock("0", 0);
}

// Block class
class Block {
    public int index;
    public String previousHash;
    public long timestamp;
    public List<Transaction> data;
    public String hash;
    public int proof;

    public Block(int index, String previousHash, long timestamp,
                 List<Transaction> data, int proof) {
        this.index = index;
        this.previousHash = previousHash;
        this.timestamp = timestamp;
        this.data = data;
        this.proof = proof;
    }
}

// Transaction class
class Transaction {
    public String sender;
    public String recipient;
    public double amount;

    public Transaction(String sender, String recipient, double amount) {
        this.sender = sender;
        this.recipient = recipient;
        this.amount = amount;
    }

    public String toString() {
        return sender + recipient + amount;
    }
}

// Add transaction
public int newTransaction(String sender, String recipient, double amount) {
    currentTransactions.add(new Transaction(sender, recipient, amount));
    return this.chain.size() + 1;
}

// Create new block
public Block newBlock(String previousHash, int proof) {
    Block block = new Block(
            this.chain.size() + 1,
            previousHash,
```

```
            new Date().getTime(),
            new ArrayList<>(this.currentTransactions),
            proof
    );

    // Generate hash AFTER block creation
    block.hash = hashBlock(block);

    // Clear transactions
    this.currentTransactions.clear();

    // Add to chain
    this.chain.add(block);

    return block;
}

// Hash block
public String hashBlock(Block block) {
    String dataString = block.data.toString();

    String blockString = block.index +
            block.previousHash +
            block.timestamp +
            dataString +
            block.proof;

    return sha256(blockString);
}

// Proof of Work
public int proofOfWork(String previousHash) {
    int proof = 0;
    while (!validProof(previousHash, proof)) {
        proof++;
    }
    return proof;
}

// Validate proof
public boolean validProof(String previousHash, int proof) {
    String guess = previousHash + proof;
    String guessHash = sha256(guess);
    return guessHash.startsWith("0000");
}

// SHA-256 hashing
public String sha256(String input) {
```

```
try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = digest.digest(input.getBytes());

        StringBuilder hexString = new StringBuilder();
        for (byte b : hashBytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();

    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}

// Get last block
public Block lastBlock() {
    return this.chain.get(this.chain.size() - 1);
}

// Validate blockchain
public boolean isChainValid() {
    for (int i = 1; i < chain.size(); i++) {
        Block current = chain.get(i);
        Block previous = chain.get(i - 1);

        if (!current.previousHash.equals(previous.hash)) {
            return false;
        }

        if (!hashBlock(current).equals(current.hash)) {
            return false;
        }
    }
    return true;
}

// Main method
public static void main(String[] args) {
    Blockchain blockchain = new Blockchain();

    // Add transactions
    blockchain.newTransaction("Alice", "Bob", 10.5);
    blockchain.newTransaction("Bob", "Charlie", 20.0);

    // Mine new block
    String previousHash = blockchain.lastBlock().hash;
    int proof = blockchain.proofOfWork(previousHash);
```

```
blockchain.newBlock(previousHash, proof);

        // Print blockchain
        for (Block block : blockchain.chain) {
            System.out.println("\nBlock #" + block.index);
            System.out.println("Hash: " + block.hash);
            System.out.println("Previous Hash: " + block.previousHash);

            for (Transaction txn : block.data) {
                System.out.println("Transaction: " +
                        txn.sender + " -> " +
                        txn.recipient + " : " +
                        txn.amount);
            }
        }

        // Validate chain
        System.out.println("\nBlockchain Valid: " + blockchain.isChainValid());
    }
}
```

Output:

Block #1
Hash: b089a2b203bce1b84f1d69aa7d1acce028acddb1c328f7e49f274571e215e5f8
Previous Hash: 0

Block #2
Hash: b38a8b1ef7b91c5dee90bb121bc1f740204937512b3f1d512ccbcf55a77932f8
Previous Hash: b089a2b203bce1b84f1d69aa7d1acce028acddb1c328f7e49f274571e215e5f8
Transaction: Alice -> Bob : 10.5
Transaction: Bob -> Charlie : 20.0

Blockchain Valid: true

=== Code Execution Successful ===

Result:

Thus, the program successfully simulated the fundamental working of a blockchain network, including transaction handling, block creation, hashing, and chain validation.

Ex.No: 4 Date:	Deploy an asset-transfer app using blockchain. Learn app development within a Hyperledger Fabric network.

Aim:

To deploy an asset-transfer application using the Hyperledger Fabric blockchain network..

Algorithm:

1. **Setup Hyperledger Fabric Network:**
   - Download and install Hyperledger Fabric samples, binaries, and Docker images.
   - Set up the network using Docker Compose.
   - Create a channel to communicate between peers.
   - Start peers and orderers in Docker containers.
2. **Write Chaincode (Smart Contract) for Asset Transfer:**
   - Develop chaincode in Java to handle asset transfer logic:
     - Initialize assets in the ledger.
     - Transfer assets from one participant to another.
     - Validate asset transfer (e.g., ensuring enough balance exists).
3. **Deploy the Chaincode:**
   - Install the chaincode on peers.
   - Instantiate the chaincode on the channel.
4. **Create Java Application for Asset Transfer:**
   - Develop a Java application using the Hyperledger Fabric SDK to interact with the network.
     - Connect to the network using a connection profile.
     - Use SDK APIs to invoke asset transfer transactions and query assets.
5. **Submit Transactions:**
   - Use the application to invoke asset transfer transactions.
   - Query the ledger to validate updated balances and confirm transactions.
6. **Verify Asset Transfer:**
   - After each transaction, query the ledger to check if the asset was transferred successfully.

Program for Hyperledger Fabric Asset Transfer

```
import org.hyperledger.fabric.shim.ChaincodeBase;
import org.hyperledger.fabric.shim.ChaincodeStub;

import java.nio.charset.StandardCharsets;
import java.util.List;

public class AssetTransfer extends ChaincodeBase {

    @Override
    public Response init(ChaincodeStub stub) {

        // Initialize ledger with assets
        String asset1 = "{\"id\":\"asset1\", \"owner\":\"Alice\", \"value\":100}";
        String asset2 = "{\"id\":\"asset2\", \"owner\":\"Bob\", \"value\":50}";

        stub.putState("asset1", asset1.getBytes(StandardCharsets.UTF_8));
        stub.putState("asset2", asset2.getBytes(StandardCharsets.UTF_8));

        return newSuccessResponse("Ledger Initialized");
    }

    @Override
    public Response invoke(ChaincodeStub stub) {

        String function = stub.getFunction();
        List<String> args = stub.getParameters();

        switch (function) {
            case "transferAsset":
                return transferAsset(stub, args);
            case "queryAsset":
                return queryAsset(stub, args);
            default:
                return newErrorResponse("Invalid function name");
        }
    }

    // Transfer asset ownership
    private Response transferAsset(ChaincodeStub stub, List<String> args) {

        if (args.size() != 2) {
            return newErrorResponse("Expecting assetId and newOwner");
        }

        String assetId = args.get(0);
        String newOwner = args.get(1);
```

```
byte[] assetBytes = stub.getState(assetId);

        if (assetBytes == null || assetBytes.length == 0) {
            return newErrorResponse("Asset not found");
        }

        String asset = new String(assetBytes, StandardCharsets.UTF_8);

        // Update owner safely
        String updatedAsset = asset.replaceAll(
                "\"owner\":\"[^\"]*\"",
                "\"owner\":\"" + newOwner + "\""
        );

        stub.putState(assetId, updatedAsset.getBytes(StandardCharsets.UTF_8));

        return newSuccessResponse("Asset transferred successfully");
    }

    // Query asset
    private Response queryAsset(ChaincodeStub stub, List<String> args) {

        if (args.size() != 1) {
            return newErrorResponse("Expecting assetId");
        }

        String assetId = args.get(0);

        byte[] assetBytes = stub.getState(assetId);

        if (assetBytes == null || assetBytes.length == 0) {
            return newErrorResponse("Asset not found");
        }

        return newSuccessResponse(assetBytes);
    }

    public static void main(String[] args) {
        new AssetTransfer().start(args);
    }
}
```

Java Application to Interact with Fabric Network

```
import org.hyperledger.fabric.gateway.*;

import java.nio.file.Path;
import java.nio.file.Paths;
```

```
public class AssetTransferApp {

    private static final String CHANNEL_NAME = "mychannel";
    private static final String CHAINCODE_NAME = "assettransfer";
    private static final String USER_NAME = "user1";

    public static void main(String[] args) {

        try {
            // Load wallet and connection profile
            Path walletPath = Paths.get("wallet");
            Path networkConfigPath = Paths.get("connection.json");

            Gateway.Builder builder = Gateway.createBuilder()
                    .identity(walletPath, USER_NAME)
                    .networkConfig(networkConfigPath)
                    .discovery(true);

            // Connect to Fabric Gateway
            try (Gateway gateway = builder.connect()) {

                Network network = gateway.getNetwork(CHANNEL_NAME);
                Contract contract = network.getContract(CHAINCODE_NAME);

                // Transfer asset
                System.out.println("Submitting transaction...");
                contract.submitTransaction("transferAsset", "asset1", "Charlie");

                // Query asset
                System.out.println("Querying asset...");
                byte[] result = contract.evaluateTransaction("queryAsset", "asset1");

                System.out.println("Result: " + new String(result));

            }

        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

REQUIRED DEPENDENCIES (pom.xml)

```
<dependencies>

   <!-- Fabric Gateway -->
   <dependency>
      <groupId>org.hyperledger.fabric</groupId>
      <artifactId>fabric-gateway-java</artifactId>
      <version>2.2.0</version>
   </dependency>

   <!-- Chaincode Shim -->
   <dependency>
      <groupId>org.hyperledger.fabric-chaincode-java</groupId>
      <artifactId>fabric-chaincode-shim</artifactId>
      <version>2.2.0</version>
   </dependency>

</dependencies>
```

Output:

Submitting transaction...
Querying asset...
Result: {"id":"asset1","owner":"Charlie","value":100}

Result :

Thus, the implementation of the Hyperledger Fabric-based asset transfer application was completed successfully

Ex.No: 5 Date:	Fitness Club rewards using Hyperledger Fabric

Aim:

To build a web app that tracks fitness club rewards using Hyperledger Fabric

Procedure:

1. Set up Hyperledger Fabric network.
2. Define smart contracts for handling rewards.
3. Develop a web application frontend.
4. Connect the frontend to the Hyperledger Fabric network

1. Set up Hyperledger Fabric network:

You need to set up a Hyperledger Fabric network with a few nodes. Refer to the official documentation for detailed instructions.

2. Define smart contracts:

Define smart contracts to handle fitness club rewards. Here's a simple example in Go:

```
package main
import (
    "encoding/json"
    "fmt"
    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)
type RewardsContract struct {
    contractapi.Contract
}
type Reward struct {
    MemberID string `json:"memberID"`
    Points   int    `json:"points"`
}
func (rc *RewardsContract) IssueReward(ctx contractapi.TransactionContextInterface, memberID string,
points int) error {
    reward := Reward{
```

```
        MemberID: memberID,
        Points:   points,
    }
    rewardJSON, err := json.Marshal(reward)
    if err != nil {
        return err
    }
    return ctx.GetStub().PutState(memberID, rewardJSON)
}
func (rc *RewardsContract) GetReward(ctx contractapi.TransactionContextInterface, memberID string)
(*Reward, error) {
    rewardJSON, err := ctx.GetStub().GetState(memberID)
    if err != nil {
        return nil, err
    }
    if rewardJSON == nil {
        return nil, fmt.Errorf("reward for member %s not found", memberID)
    }
    var reward Reward
    err = json.Unmarshal(rewardJSON, &reward)
    if err != nil {
        return nil, err
    }
    return &reward, nil
}
```

3. Develop a web application frontend:

You can use any frontend framework like React, Vue.js, etc. Here's a simple React component to interact with the smart contract:

RewardsComponent.js

```
import React, { useState } from 'react';
import { useContract } from './useContract'; // Assume this hook connects to the contract
const RewardsComponent = () => {
```

```
  const [memberID, setMemberID] = useState('');
  const [points, setPoints] = useState('');
  const { issueReward, getReward } = useContract();
  const handleIssueReward = async () => {
    await issueReward(memberID, points);
  };
  const handleGetReward = async () => {
    const reward = await getReward(memberID);
    console.log(reward);
  };
  return (
    <div>
      <input type="text" placeholder="Member ID" value={memberID} onChange={(e) =>
setMemberID(e.target.value)} />
      <input type="number" placeholder="Points" value={points} onChange={(e) =>
setPoints(e.target.value)} />
      <button onClick={handleIssueReward}>Issue Reward</button>
      <button onClick={handleGetReward}>Get Reward</button>
    </div>
  );
};
export default RewardsComponent;
```

4. Connect the frontend to the Hyperledger Fabric network:

Use a library like fabric-network to interact with the Hyperledger Fabric network. Implement functions like issueReward and getReward to interact with the smart contract.

Now, integrate this frontend component into your web application. Here's a screenshot of what the UI might look like:

In this example, users can input a member ID and points to issue rewards, and they can retrieve rewards by providing the member ID.

Remember, this is a basic example. In a real-world application, you would need to consider security, scalability, and other factors. Additionally, you'll need to handle user authentication, authorization, and other functionalities as per your requirements.

Result:

Thus, the blockchain to track fitness club rewards and build a web app that uses Hyperledger Fabric to track and trace member rewards are executed successfully.

Ex.No: 6 Date:	Car auction network using the Hyperledger fabric node SDK

Aim:

To create a "Hello World" example of a car auction network using the Hyperledger Fabric Node SDK and IBM Blockchain Starter Plan

Procedure:

1. Set up a Hyperledger Fabric network using the IBM Blockchain Starter Plan.
2. Define and deploy chaincode (smart contract) for the car auction.
3. Develop a Node.js application using the Hyperledger Fabric Node SDK to interact with the chaincode.
4. Implement basic functionalities such as creating an auction, bidding. and retrieving auction results.

1. Set up a Hyperledger Fabric network using IBM Blockchain Starter Plan:

Follow the instructions provided by IBM to set up a Hyperledger Fabric network using the IBM Blockchain Starter Plan.

2. Define and deploy chaincode:

Define chaincode for the car auction network. This chaincode will contain functions to create auctions, place bids, and retrieve auction results.

Program:

```
package main
import (
   "encoding/json"
   "fmt"
   "github.com/hyperledger/fabric-contract-api-go/contractapi"
)
type CarAuctionContract struct {
   contractapi.Contract
}

 type Auction struct {
   ID         string `json:"id"`
   Car        string `json:"car"`
   HighestBid int    `json:"highestBid"`
```

```
    HighestBidder string `json:"highestBidder"`
}

func (c *CarAuctionContract) CreateAuction(ctx contractapi.TransactionContextInterface, auctionID
string, car string) error {
    auction := Auction{
        ID:    auctionID,
        Car:   car,
    }
    auctionJSON, err := json.Marshal(auction)
    if err != nil {
        return err
    }
    return ctx.GetStub().PutState(auctionID, auctionJSON)
}

func (c *CarAuctionContract) PlaceBid(ctx contractapi.TransactionContextInterface, auctionID string,
bidder string, bidAmount int) error {
    auctionJSON, err := ctx.GetStub().GetState(auctionID)
    if err != nil {
        return err
    }
    if auctionJSON == nil {
        return fmt.Errorf("auction %s does not exist", auctionID)
    }

    var auction Auction
    err = json.Unmarshal(auctionJSON, &auction)
    if err != nil {
        return err
    }
    if bidAmount > auction.HighestBid {
        auction.HighestBid = bidAmount
```

```
        auction.HighestBidder = bidder
        updatedAuctionJSON, _ := json.Marshal(auction)
        return ctx.GetStub().PutState(auctionID, updatedAuctionJSON)
    }
    return fmt.Errorf("bid amount should be higher than the current highest bid")
}
func (c *CarAuctionContract) GetAuction(ctx contractapi.TransactionContextInterface, auctionID string)
(*Auction, error) {
    auctionJSON, err := ctx.GetStub().GetState(auctionID)
    if err != nil {
        return nil, err
    }
    if auctionJSON == nil {
        return nil, fmt.Errorf("auction %s does not exist", auctionID)
    }
    var auction Auction
    err = json.Unmarshal(auctionJSON, &auction)
    if err != nil {
        return nil, err
    }
    return &auction, nil
}
```

3. Develop a Node.js application:

Use the Hyperledger Fabric Node SDK to interact with the chaincode. This application will have functions to create auctions, place bids, and retrieve auction results.

```
// app.js
const { Gateway, Wallets } = require('fabric-network');
const path = require('path');
const fs = require('fs');
const ccpPath = path.resolve(__dirname, '..', 'connection.json');
const ccpJSON = fs.readFileSync(ccpPath, 'utf8');
```

```
const ccp = JSON.parse(ccpJSON);
async function main() {
  try {
    const walletPath = path.join(process.cwd(), 'wallet');
    const wallet = await Wallets.newFileSystemWallet(walletPath);
    const gateway = new Gateway();
    await gateway.connect(ccp, {
      wallet,
      identity: 'user1',
      discovery: { enabled: true, asLocalhost: true }
    });
    const network = await gateway.getNetwork('mychannel');
    const contract = network.getContract('carAuction');
    // Invoke chaincode functions here
    await gateway.disconnect();
  } catch (error) {
    console.error(`Failed to submit transaction: S{error}`);
    process.exit(1);
  }
}
main();
```

4. Implement basic functionalities:

Implement functions in `app.js` to interact with the chaincode.

```
async function createAuction(auctionID, car) {
  await contract.submitTransaction('CreateAuction', auctionID, car);
  console.log(`Auction S{auctionID} created for car S{car}`);
}
async function placeBid(auctionID, bidder, bidAmount) {
```

```
    await contract.submitTransaction('PlaceBid', auctionID, bidder, bidAmount);
    console.log(`Bid placed for auction ${auctionID} by ${bidder} with amount ${bidAmount}`);
}
async function getAuction(auctionID) {
    const auctionJSON = await contract.evaluateTransaction('GetAuction', auctionID);
    const auction = JSON.parse(auctionJSON.toString());
    console.log(`Auction ${auctionID}: ${JSON.stringify(auction, null, 2)}`);
}
```

Output:

```
Auction CAR123 created for car BMW
Bid placed for auction CAR123 by bidder1 with amount 5000
Auction CAR123: {
  "id": "CAR123",
  "car": "BMW",
  "highestBid": 5000,
  "highestBidder": "bidder1"
}
```

Result:

Thus the creation of "Hello World" example of a car auction network using the Hyperledger Fabric Node SDK and IBM Blockchain Starter Plan are executed successfully.